

Programación Orientada a Objetos 2

Diagrama de clases

Daniela Carvajal Espinosa

1040323319

Maria Alejandra Correa Berrio

1193054964

Sandra Vanessa Roldan Monsalve

1017178663

Profesor

Boris Alberto Salleg

Institución Universitaria Digital de Antioquia

Medellín

Facultad de Ingeniería y Ciencias Agropecuarias

Pregrado Tecnología en Desarrollo de Software

Domingo 22 de marzo de 2026

Tabla de contenido

Introducción.-----	3
Objetivo general-----	4
Objetivos específicos.-----	4
Desarrollo de la actividad.-----	5
Descripción de las Clases Identificadas.-----	5
Justificación de Relaciones de UML y Multiplicidades.-----	6
Principios de Diseño: Cohesión, acoplamiento y SOLID.. -----	6
Enlaces. -----	9
Conclusiones.-----	10

Introducción

El lenguaje del Modelo Unificado (UML) es una herramienta fundamental en el desarrollo de software, ya que esto nos permite estructurar y comunicar visualmente cómo va a funcionar un programa antes de escribir la primera línea de código. especialmente los diagramas de clase, nos ayuda a organizar la arquitectura estática de un sistema orientado a objetos, definiendo qué elementos lo componen y cómo interactúan entre sí. En este trabajo diseñamos el modelo para un sistema de biblioteca, donde es necesario gestionar de una forma eficiente recursos como libros físicos y digitales, los autores, los usuarios y las transacciones de préstamos. El propósito principal de este proyecto no es sólo aplicar la teoría, sino también experimentar la dinámica del trabajo colaborativo real. Para lograrlo utilizamos plataformas de diagramación para el modelo visual y repositorios de GitHub para organizar nuestras versiones, simulando las prácticas normales de un entorno de desarrollo profesional.

Objetivo General

- Diseñar un diagrama de clase UML para un sistema de biblioteca que aplique los conceptos avanzados de la Programación Orientada a Objeto, garantizando un modelo funcional, escalable y bien estructurado.

Objetivos Específicos

- Aplicar la abstracción y el encapsulamiento para proteger y organizar la información de cada entidad del sistema.
- Implementar relaciones de herencia y polimorfismo para promover la reutilización de código entre los diferentes tipos de recursos.
- Utilizar los principios de SOLID para asegurar que el sistema mantenga una cohesión alta y un bajo acoplamiento en sus componentes.

Desarrollo de la actividad

- Para construir el diagrama empezando analizando qué entidades eran indispensables para que la biblioteca funcionara correctamente y como debían agruparse.

Descripción de las clases identificadas

Definimos como libro la clase base, ya que encapsula las propiedades generales del catálogo (como el título y su disponibilidad) y arroja métodos de gestión cómo esDisponible(), prestar() y devolver (). De esta base derivan las subclases: LibroDigital y LibroFísico. La versión digital añade atributos específicos como el formato y el tamaño, además de la acción descargar (). Por su parte la versión física gestiona su ubicaciónEstante. También incluimos la clase Autor, que almacena los datos del creador y gestiona sus publicaciones mediante agregarLibro()

El núcleo organizativo recae en la clase biblioteca, una entidad contenedora que ofrece servicios globales para registrar y buscar libros, finalmente, modelamos al Usuario (el cliente que realiza las acciones) y a la clase Préstamo, que funciona como una entidad transaccional encargada de registrar las fechas de inicio y devolución de los ejemplares solicitados.

Justificación de las Relaciones de UML y Multiplicidades

Luego de definir las clases, establecimos sus interacciones. Aplicamos la herencia desde LibroDigital y LibroFísico hacia Libro, lo que nos permitió aprovechar la estructura base y usar el polimorfismo al ejecutar métodos compartidos con el de prestar. Para la agregación, conectamos a la Biblioteca y al Autor con los libros (relación de 1 a 0..*); decidimos usarla porque representa una contención débil: si se elimina un autor o una sucursal, los libros como concepto siguen existiendo independientemente.

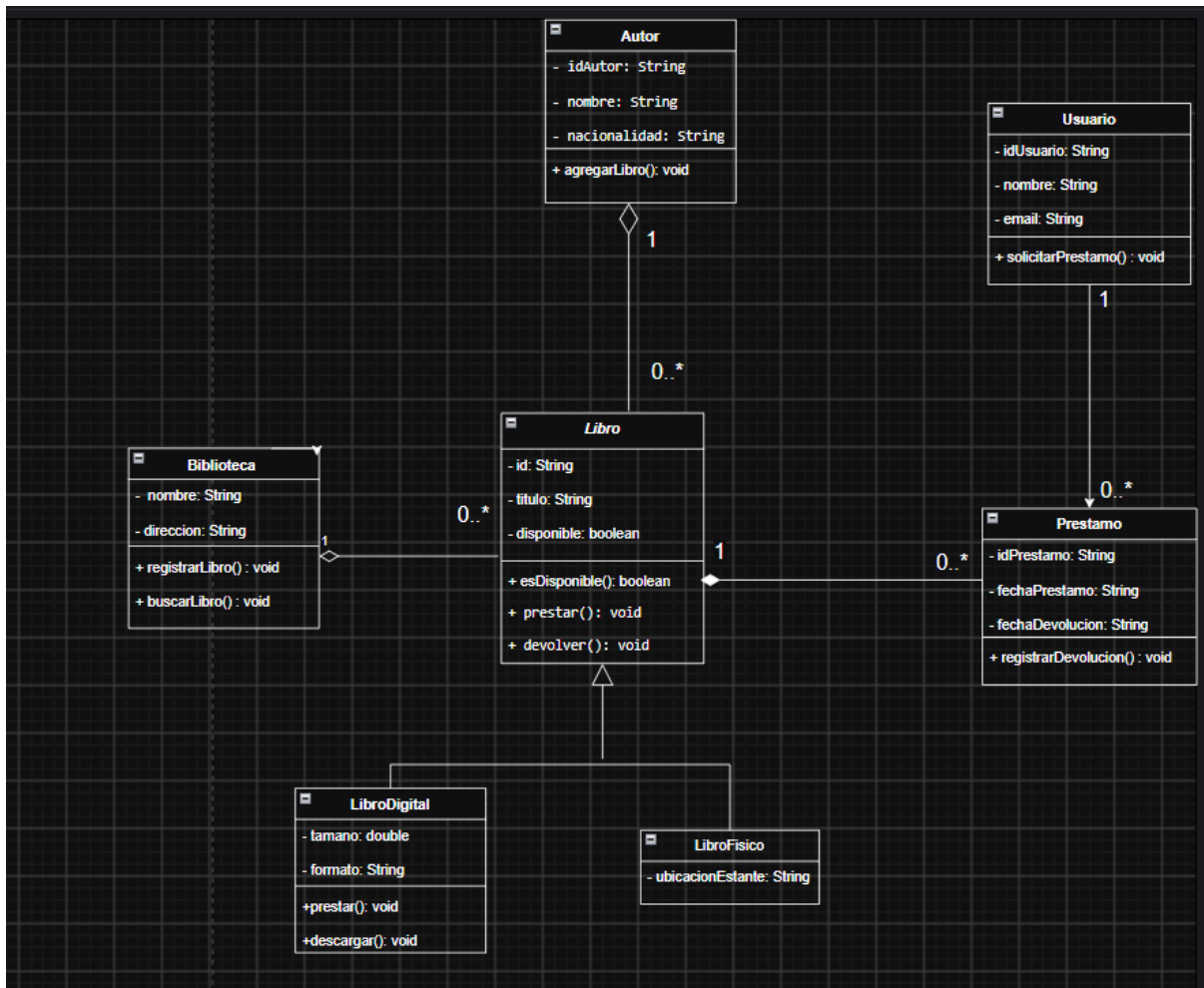
Por otro lado, utilizamos la composición Préstamo y Libro (*a 1), dado a que es una dependencia muy fuerte; el registro de un préstamo pierde todo su sentido si no existe la referencia libro prestado. Por último, marcamos una asociación simple entre Usuario y Préstamo (1 a 0..*) para ilustrar de forma directa que una persona puede realizar múltiples solicitudes de libros a lo largo del tiempo.

Principios de Diseño: Cohesión, acoplamiento y SOLID

Durante el diseño, procuramos mantener una alta cohesión, asegurando que cada clase tuviera un propósito único y delimitado. Al mismo tiempo, minimizamos la dependencia directa entre ellas (buscando un bajo acoplamiento), permitiendo que interactuaran únicamente a través de sus métodos públicos.

Este enfoque se respalda aplicando los principios SOLID:

- **Responsabilidad única:** La clase Préstamo. Administra exclusivamente los plazos de la transacción, delegando el manejo del estado físico del objeto a la clase Libro.
- **Abierto/Cerrado (OCP):** La estructura del diseño nos permite agregar nuevos tipos de publicaciones a futuro (como un audiolibro) simplemente heredado de la clase Libro, sin necesidad de alterar el código del sistema ya existente.
- **Sustitución de Liskov (LSP):** Nos aseguramos de que las instancias de LibroDigital y LibroFísico puedan ser tratadas genéticamente como un “Libro” sin alterar el comportamiento esperado en las transacciones de la biblioteca.



Enlaces

Enlace del diagrama

https://app.diagrams.net/#G1x-cB1xzix6heeBKJefF_HgZ-JbI5VxbI#%7B%22pageId%22%3A%22wgpdqHHdWUw9IUFqizby%22%7D

Enlace del repositorio

Enlace del video

https://drive.google.com/file/d/1gTZyobBQO0RXgch_-wveAdXF32VTlink/view?usp=sharing

[g](#)

Conclusiones Personales

- A nivel personal esta actividad me ayudó a comprender conceptos que antes, para mí sentía en otro idioma. Ver tantas flechas, símbolos y reglas técnicas me abrumó un poco, pero al ir estructurando el modelo de biblioteca, todo empezó a cobrar sentido lógico.
- Entender en la práctica cómo una clase hereda de otra o por qué es tan importante proteger los datos con el encapsulamiento, me hizo ver que la Programación Orientada a Objetos no se trata solo de escribir líneas de código en automático, sino de saber planear y estructurar lógicamente las cosas antes de construirlas.
- Trabajar en equipo y debatir si debíamos usar una agregación o una composición demostró lo mucho que aún me falta por aprender, pero a la vez me dio tranquilidad. Equivocarme, borrar y corregir este diagrama me acercó un poco más a la realidad de lo que hace un Desarrollador, demostrándome que voy dando los pasos correctos para mi vida profesional.
- Realizar este trabajo me permitió ver que la programación no es solamente códigos, que también es de mucha lógica y razonamiento, entender de dónde salen todas las cosas, sus porqués y para qué es fundamental para que la estructura del código tenga un sentido lógico.
- Entender la herencia y el polimorfismo, me pareció fundamental, debido a que con ello evitamos duplicar cosas que están repetidas, es decir, evitamos duplicar procesos y así volver el código más eficiente.
- Manejar GitHub fue un poco difícil para mi persona, debido a que mi persona Alejandra no había tenido acercamiento a este, pero gracias a que mis compañeras Vanessa y Daniela ya habían tenido acercamiento, su acompañamiento, apoyo y explicación fue fundamental para aprender a crear los repositorios.